Cyber Developer Program

Java Secure Coding Foundations

Day 7 Exercise Lab: Vulnerability Scanner & Risk Register Toolkit

Lab Type	Submission	Grading Scale
Individual, graded	Source files, per instructor process	0–4 per criterion (see rubric)

Scenario

The security team's Risk Register needs to grow up. It's no longer enough to hold findings in memory during a single run — it needs to persist between sessions, reject bad data on the way in, and absorb a batch of archived findings from the security team's old scanning tool. This afternoon, you'll take everything from this morning (unit testing, secure TDD, debugging, and file I/O) and apply it to the same security-tool narrative you've been building all week.

Before You Start

Bring forward your Day 5 files if you have them, or use the reference copies included in this lab folder:

- Vulnerability.java
- Asset.java
- Severity.java

You're also given LegacyScanOutputParser.java, which converts one line of the old scanner's output format into a Vulnerability. It has already been written and tested — you shouldn't need to modify it unless your instructor tells you otherwise.

Two more files are provided complete, ready to use as-is:

- RiskRegisterApp.java — a small runnable console app for exercising your RiskRegister by hand once it compiles. Run it, add a few findings, save and reload a file, and try importing a legacy scan file, to sanity-check your work before (and while) you write your test suite.
- RiskRegisterTest.java — this is NOT complete. It's a fill-in skeleton: every required test already has a name, a method signature, and a comment describing exactly what it must verify, but the body is just fail("TODO: ..."). Your job is to replace each fail(...) call with real Arrange-Act-Assert code. Running the suite as given will show 11 honest failures — that's expected, and it's how you'll know which tests still need work.

What You're Building

Your RiskRegister class must include all of the following pieces:

1. Core Register Logic

A List<Vulnerability> holding every finding, and a Set<String> tracking every unique CVE ID currently in the register. addFinding() validates a finding before accepting it, and rejects an attempt to add a CVE ID that's already present.

2. Secure Input Validation

Two validators, applying this morning's edge-case catalog to two different fields:

- validateCveId(String) — must match CVE-YYYY-NNNN (a 4-digit year, then a 4-7 digit sequence number). Covers null, blank, malformed, and overflow in a single check.
- validateComponent(String) — not null/blank, no more than 50 characters, and free of '|' and line breaks (these are your save file's delimiters — an unvalidated component name is an injection path into your own file format).

3. File I/O — Your Own Format

saveToFile(String) writes one pipe-delimited line per finding:

```
cveId|component|version|severity|estimatedRemediationCost|discoveredDate
CVE-2024-1234|openssl|3.0.1|CRITICAL|150.00|2024-01-15
```

loadFromFile(String) reads that format back, rejecting (with InvalidRegisterDataException) any line that doesn't split into exactly 6 parts, or where any part fails to parse.

4. File I/O — Legacy Import

importLegacyScanResults(String) reads an old-format scan dump, uses LegacyScanOutputParser.parseLine() on each line, and routes every result through the SAME addFinding() validation as everything else — legacy data is still external input the moment it comes from a file.

5. A Real Test Suite

Complete the provided RiskRegisterTest.java skeleton, covering, at minimum:

- Happy path: adding a valid finding; a full save/load round trip
- Boundary-value analysis: a CVE ID at exactly the maximum sequence length (accepted) and one digit past it (rejected)
- Abuse cases for both validators, covering the edge-case catalog (null, empty, malformed, injection-shaped)
- Duplicate CVE ID handling, and that getFindings()/getUniqueCveIds() can't be mutated externally
- File I/O abuse cases: a missing file, and a corrupted save file

Grading Focus

- Correct List/Set usage and encapsulation (no leaked mutable references)
- Secure, edge-case-catalog-driven validation on both the CVE ID and component fields
- Correct core register logic, including duplicate detection
- Working, safe file I/O for both your own format and the legacy import path
- A test suite that covers happy-path, boundary-value, and abuse cases — not just the happy path

Suggested Approach

1. Confirm your Day 5 Vulnerability.java and Asset.java compile on their own (or use the reference copies provided).

2. Build validateCveId() and validateComponent() first, and write a few quick manual checks (or early tests) before moving on.
3. Build addFinding() and the encapsulated getters. Write happy-path and abuse-case tests for this before moving to file I/O.
4. Build saveToFile() and loadFromFile() for your own format. Test the round trip, then test a corrupted file.
5. Build importLegacyScanResults() last, using the provided parser.
6. Run RiskRegisterApp.java at any point once your code compiles — it's a fast way to manually confirm things behave as expected before you rely on your test suite.
7. Fill in RiskRegisterTest.java one method at a time, replacing each fail("TODO: ...") with real Arrange-Act-Assert code, rather than leaving all 11 for the end.
8. Keep an eye out for anything your instructor announces mid-lab — today's debugging skills may come in handy.

Common Pitfalls

- Checking for duplicates before validating the new finding, instead of after — an invalid CVE ID should never even reach the duplicate check
- Using < or > instead of != when checking that a split line has the exact number of fields you expect
- Returning a live reference to your internal List or Set from a getter
- Writing only happy-path tests and assuming the suite is “done”

Submission

Submit your completed .java files following your instructor's standard submission process. Make sure your project compiles from a clean directory, that RiskRegisterApp.java runs without crashing, and that your completed test suite runs with zero remaining fail("TODO: ...") calls and zero failures, before you submit.